

Loading States

Visual stability means that a page's layout remains predictable and free of unexpected shifts as content loads. Loading states help achieve this and provide users with clear visual feedback about ongoing asynchronous operations, such as data fetching. React Router exposes two distinct mechanisms for managing loading states: declarative and imperative. Both mechanisms serve the same user-facing goal, communicating that an operation is in progress. They differ in where and how the loading state is defined and consumed. The appropriate choice depends on the scope and trigger of the async operation.

Note

This documentation builds on [Data Fetching](#). In the context of classifying data as critical and non-critical, the term visual stability was mentioned in [Data Fetching](#), along with its relevance to user-perceived, or psychological, performance characteristics of a web app.

Contents

[Declarative Loading States](#)[Imperative Loading States](#)[Visual Feedback Patterns](#)

Declarative Loading States

Declarative loading states are defined structurally in the component tree via React's `<Suspense/>` [↗](#) component.

`Suspense` acts as an asynchronous boundary: it intercepts rendering of any child component that isn't yet ready and renders a `fallback` UI in its place until the operation completes. A child component signals that it's not ready by throwing a Promise. Native React concepts, such as the `use()` [↗](#) hook, implement that convention, but so do React Router's built-in [data streaming](#) [↗](#) or `<Await/>` [↗](#) patterns.

`Suspense` integrates with React's concurrent rendering model, which enables React to pause and resume rendering work without blocking the main thread. This is the foundational mechanism that enables both visual stability and responsive loading states in modern React apps.

Loading State Fundamentals

To opt into a declarative loading state, a route loader returns a Promise without awaiting it. React Router exposes this unresolved Promise via the route's loader data, which is then passed downstream for resolution in the component tree. A `Suspense` boundary renders the fallback until that Promise settles. Critical data is awaited in the loader and available



```
1 // route.tsx
2 export async function loader() {
3   const critical = await fetchCriticalData();
4   const nonCritical = fetchNonCriticalData(); // int
5   return { critical, nonCritical };
6 }
7
8 export default function MyRoute() {
9   const { critical, nonCritical } = useLoaderData();
10  return <Suspense fallback={<Skeleton />}>{/* ... */
11 }
```

The use() and Suspense Pattern

React 19 introduced the `use()` hook, which reads a Promise directly inside a component. When the Promise isn't yet settled, `use()` suspends the component, triggering the nearest `Suspense` boundary without requiring any additional abstraction.

```
1 function NonCriticalContent({ promise }) {
2   const data = use(promise);
3   return <div>{data}</div>;
4 }
5
6 <Suspense fallback={<Skeleton />}>
7   <NonCriticalContent promise={nonCritical} />
8 </Suspense>;
```

Note

While `use()` is portable and framework-agnostic, it has two relevant limitations in this context: it requires the consuming logic to be extracted into a separate component, and provides no built-in mechanism for co-locating error handling with the loading boundary.

The Await and Suspense Pattern

React Router's `Await` component addresses both limitations. It resolves the Promise inline via a render prop, eliminating the need for a separate component, and accepts an optional `errorElement` prop that catches rejections locally, keeping the loading and error boundary co-located with the async operation. For these reasons, `Await` is the recommended pattern when working within React Router's data layer.

```
1 <Suspense fallback={<Skeleton />}>
2   <Await resolve={nonCritical}>{(resolved) => <NonCr
3 </Suspense>
```

When a Promise passed to `Await` rejects, the error propagates to the nearest error boundary in the tree. For more granular



```
1 <Suspense fallback={<Skeleton />}>
2   <Await resolve={nonCritical} errorElement={<ErrorF
3     {(resolved) => <NonCriticalContent data={resolve
4   </Await>
5 </Suspense>
```

Suspense Boundary Granularity

Multiple independent `<Suspense>` boundaries can coexist within a single route, each encapsulating one async operation. Conceptually, each `<Suspense>` boundary acts like a `try/catch` for async. Everything inside the boundary is treated as a single loading unit, and content outside the boundary isn't affected by the suspensions within it.

This is why each promise **must** be wrapped in its own `<Suspense>` boundary when a component consumes multiple asynchronous data sources. This applies to React Router's `<Await>`, React's `use()`, and any other suspending mechanism. If multiple promises share a single boundary, the faster promise resolves first and triggers a re-render. The still-pending slower promise then suspends the boundary again. Already-rendered content is torn down, and the fallback reappears. The result is visible content flicker and [layout shift](#), which inflates interactivity metrics like [Interaction to Next Paint](#) (INP).

With separate boundaries, each promise resolves independently. Fast content streams in and stays visible. Slower content continues loading with its own skeleton. For more details, see the React 18 architecture discussion on GitHub: [New Suspense SSR Architecture in React 18](#).

Correct: each Await has its own Suspense boundary

```
</>
1 <>
2   <Suspense fallback={<SkeletonA />}>
3     <Await resolve={promiseA}>{(resolved) => <Compon
4   </Suspense>
5
6   <Suspense fallback={<SkeletonB />}>
7     <Await resolve={promiseB}>{(resolved) => <Compon
8   </Suspense>
9 </>
```

Correct: use() wrapper components, each behind its own Suspense

```
</>
1 function ProductWrapper({ promise }: { promise: Prom
2   const product = use(promise);
3   return <ProductContent product={product} />;
4 }
5
6 function ReviewsWrapper({ promise }: { promise: Prom
```

[Try for free](#)

```
12 <Suspense fallback={<ProductSkeleton />}>
13   <ProductWrapper promise={productPromise} />
14 </Suspense>
15 <Suspense fallback={<ReviewsSkeleton />}>
16   <ReviewsWrapper promise={reviewsPromise} />
17 </Suspense>
18 </div>;
```

Anti-pattern: multiple promises sharing one boundary

```
</> | |
1 // ❌ BAD: Both Await children share a single Suspense boundary
2 // When promise1 resolves first, Component1 renders
3 // promise2 (still pending) suspends the boundary again
4 // down Component1 and showing <ComponentsSkeleton />
5 <Suspense fallback={<ComponentsSkeleton />}>
6   <div>
7     <Await resolve={promise1}>{(resolved) => <Component1 />}
8     <Await resolve={promise2}>{(resolved) => <Component2 />}
9   </div>
10 </Suspense>
```

Anti-pattern: multiple use() calls in one component

```
</> | |
1 // ❌ BAD: Both use() calls suspend within the same
2 // Suspense boundary which means they share the nearest parent Suspense
3 // Resolution of one promise triggers a re-render then
4 // for the other, causing the same fallback thrashing
5 function CombinedView({
6   promise1,
7   promise2,
8 }) {
9   promise1: Promise<Component1Data>,
10  promise2: Promise<Component2Data>,
11 } {
12   const data1 = use(promise1);
13   const data2 = use(promise2);
14   return (
15     <>
16       <Component1 resolved={data1} />
17       <Component2 resolved={data2} />
18     </>
19   );
20 }
21
22 <Suspense fallback={<ComponentsSkeleton />}>
23   <CombinedView promise1={promise1} promise2={promise2} />
24 </Suspense>;
```

Anti-pattern: nested Await inside one boundary

```
</> | |
1 // ❌ BAD: promise2 cannot even begin to resolve until
2 // promise1 resolves, creating an artificial waterfall on top of the shared
3 // Suspense boundary
4 <Suspense fallback={<ComponentsSkeleton />}>
5   <Await resolve={promise1}>
6     {(resolved1) => (
7       <div>
8         <Component1 />
9         <Component2 />
10      </div>
11     )}
12   </Await>
13 </Suspense>
```



```
11 </Await>
12 </Suspense>
```

✓ Tip

Exception: Truly dependent promises (for example, fetching details after a list) can share a boundary because they represent one logical loading unit.

Promise Identity

A `<Suspense>` boundary identifies a pending promise by its reference, not by its value. When a child suspends, the boundary remembers *that exact promise object* and waits for it to settle. On the next render, if the child hands back a different promise object – even one that resolves to the same data – the boundary treats it as new pending work, throws away what it just rendered, and shows the fallback again. With a fresh promise on every render, the cycle never ends and the fallback flickers forever. This applies equally to React's `use()` and React Router's `<Await>`.

So the rule is: *the same logical operation must produce the same promise object across renders.*

Promises returned from a route `Loader` satisfy this automatically – React Router preserves their identity for the lifetime of the active route match, whether read via `useLoaderData`, `useRouteLoaderData`, or `useOutletContext`. Anything composed in the component body with `Promise.all`, `Promise.race`, `.then(...)`, or any wrapper expression does not: the expression is evaluated on every render, producing a brand-new Promise object each time.

Anti-pattern: composing promises in the component

```
// ✗ BAD: Promise.all returns a new promise every r
// The Suspense boundary never sees the resolved ref
function CombinedView() {
  const { promise1, promise2 } = useLoaderData();
  const combined = Promise.all([promise1, promise2])
  return (
    <Suspense fallback=<Skeleton />>
      <Await resolve={combined}>{[a, b]} => <Content
    </Suspense>
  );
}
```

`useMemo(() => Promise.all([p1, p2]), [p1, p2])` doesn't fix this: React discards the memo cache when the component suspends on initial mount (see [useMemo Caveats](#), "a state variable or a ref may be more appropriate"). Failure mode is confirmed in [remix-run/remix#7392](#).

Fix 1—Primary: Compose in the Loader



```
1 // route.tsx
2 export async function loader() {
3   const critical = await fetchCritical();
4   // Created once per request; loaderData preserves
5   const combined = Promise.all([fetchA(), fetchB()])
6   return { critical, combined };
7 }
8
9 function CombinedView() {
10   const { combined } = useLoaderData();
11   return (
12     <Suspense fallback=<Skeleton />>
13       <Await resolve={combined}>{[a, b]} => <Content />
14     </Suspense>
15   );
16 }
```

Fix 2—Primary: Split into Sibling Boundaries

If the promises don't truly form one loading unit, give each its own `<Suspense>` boundary so they resolve independently (see [Suspense Boundary Granularity](#)). No composition needed.

```
1 function CombinedView({ p1, p2 }) {
2   return (
3     <>
4       <Suspense fallback=<SkeletonA />>
5         <Await resolve={p1}>(a) => <SectionA data={a} />
6       </Suspense>
7       <Suspense fallback=<SkeletonB />>
8         <Await resolve={p2}>(b) => <SectionB data={b} />
9       </Suspense>
10    </>
11  );
12 }
```

Fix 3—Escape Hatch: Pin the Composed Promise Locally

Use only when neither primary fix applies (for example, the inputs arrive from a parent layout's `useOutletContext`, props, or fetcher hooks, and the consuming component must combine them). The examples assume `p1` and `p2` are already stable references; how they're obtained doesn't matter.

Variant A – lazy `useState` pin. Frozen for the component's lifetime; invalidate by remounting via `<Component key={inputIdentity} />`.

```
1 function CombinedView({ p1, p2 }) {
2   // Lazy initializer runs once per component lifetime
3   const [combined] = useState(() => Promise.all([p1, p2]))
4   return (
5     <Suspense fallback=<Skeleton />>
6       <Await resolve={combined}>{...}</Await>
7     </Suspense>
8   );
9 }
```



component can be re-rendered the key (for example, input change on revalidation while the component stays mounted). Repins when input identity changes. Survives Suspense throws.

```
1 function CombinedView({ p1, p2 }) {
2   const pinRef = useRef(null);
3   if (pinRef.current === null || pinRef.current.inputs !== [p1, p2]) {
4     pinRef.current = { inputs: [p1, p2], combined: Promise.all([p1, p2]) };
5   }
6   return (
7     <Suspense fallback=<Skeleton />>
8       <Await resolve={pinRef.current.combined}>{...}</Await>
9     </Suspense>
10  );
11 }
```

Note

The same rule applies to `use()` wrappers: a child that calls `use(somePromise)` must receive a stable promise reference as a prop. Constructing the promise in the parent's render (for example, `<Child promise={Promise.all([a, b])} />`, `<Child promise={a.then(transform)} />`) re-suspends on every render for the same reason.

Imperative Loading States

Imperative loading states aren't defined structurally in the component tree but are read programmatically via hooks. Rather than triggering a `Suspense` boundary, these hooks expose the current state of an async operation as a discrete value that the component consumes conditionally. This makes them the appropriate choice for scenarios where a `Suspense` boundary is either unavailable or insufficient, such as global navigation feedback, out-of-band mutations, or manual revalidation. For details on how these hooks relate to the broader state model, see [State Management](#).

Summary

Hook	Scope	State property
<code>useNavigation()</code> ↗	global / route transition	<code>navigation.state</code>
<code>useFetcher()</code> ↗	component-level	<code>fetcher.state</code>
<code>useFetchers()</code> ↗	global (all active)	<code>fetchers[].state</code>



route

`useSubmit()` component-
level

via

`useNavigation()`

The `useNavigation()` Hook

`useNavigation()` reflects the state of the current route transition. It covers the full navigation lifecycle, from when a transition is initiated until the destination route has fully loaded. `useNavigation()` is typically consumed in a root or layout component to provide a global loading indicator that covers all route transitions uniformly.



```
1  const navigation = useNavigation();
2
3  return (
4    <>
5      {navigation.state === "loading" && <GlobalSpinne
6        <Outlet />
7      </>
8    );
```

`navigation.state` exposes three values:

- `idle` – no navigation in progress
- `loading` – a route transition is in progress and its loader is running
- `submitting` – a form submission or action is in progress

The `useFetcher()` Hook

`useFetcher()` enables data fetching and action submissions outside of route transitions, without triggering a navigation. It's the appropriate mechanism for out-of-band operations such as inline mutations, background data refreshes, or optimistic UI updates. Unlike `useNavigation()`, `useFetcher()` is scoped to the specific operation it initiates, making it suitable for granular, component-level loading feedback.



```
1  const fetcher = useFetcher();
2
3  return (
4    <fetcher.Form method="post" action="/api/update">
5      <button type="submit">{fetcher.state === "submit
6    </fetcher.Form>
7  );
```

`fetcher.state` mirrors `useNavigation().state` in structure:

- `idle` – no operation in progress
- `loading` – fetching data from a loader



`useFetchers()` [↗](#) returns an array of all in-flight fetchers across the application. While `useFetcher()` is scoped to a single component, `useFetchers()` provides a global view of all active fetcher operations. This is useful for aggregate loading indicators. For example, showing a single “saving” badge when any background mutation is in progress.

```
1 import { useFetchers } from "react-router";
2
3 function GlobalSavingIndicator() {
4   const fetchers = useFetchers();
5   const isSaving = fetchers.some((f) => f.state !==
6
7   if (!isSaving) {
8     return null;
9   }
10  return <div className="saving-indicator">Saving...</
11 }
```

Each entry in the array mirrors the shape of a single `useFetcher()` instance, exposing `state`, `formData`, `data`, and other properties. The mirroring enables patterns, such as counting active operations or building optimistic lists from multiple concurrent mutations.

The `useRevalidator()` Hook

`useRevalidator()` [↗](#) manually triggers revalidation of the current route's loader data, outside the standard navigation lifecycle. It's relevant when external events invalidate the current data and a reload is required without a full navigation.

```
1 const revalidator = useRevalidator();
2
3 return (
4   <button onClick={() => revalidator.revalidate()} d
5     {revalidator.state === "loading" ? "Refreshing..
6   </button>
7 );
```

`revalidator.state` exposes two values:

- `idle` – no revalidation in progress
- `loading` – revalidation is running

The `useSubmit()` Hook

`useSubmit()` [↗](#) programmatically triggers form submissions or action calls without requiring a `<Form>` element. It doesn't expose its own state. The resulting async operation is reflected in `useNavigation().state`, which transitions to `submitting` for the duration of the action. Because `useSubmit()` delegates state to `useNavigation()`, it's typically paired with it explicitly when loading feedback is required.



```
4   return (  
5     <button  
6       onClick={() => submit({ key: "value" }, { method  
7         disabled={navigation.state === "submitting"}  
8       >  
9         {navigation.state === "submitting" ? "Submitting  
10      </button>  
11    );
```

Visual Feedback Patterns

Visual feedback patterns determine how loading states are communicated to the user. The goal is twofold: signal that an operation is in progress, and maintain visual stability so that the UI doesn't shift unexpectedly when content resolves. The appropriate pattern depends on the scope of the operation and the nature of the content being loaded.

Skeleton Screens vs. Spinners

Spinners communicate that something is happening without making any assumptions about the shape of the incoming content. They're appropriate for global or indeterminate operations where the layout of the resolved content is unknown or irrelevant.

```
1 <Suspense fallback={<Spinner />}>  
2   <Await resolve={promise}>{(resolved) => <Component  
3 </Suspense>
```

Skeleton screens reserve the approximate layout of the incoming content before it resolves. This reduces perceived loading time and prevents layout shift, as the resolved content occupies space that was already allocated.

```
1 <Suspense fallback={<CardSkeleton />}>  
2   <Await resolve={promise}>{(resolved) => <Card data  
3 </Suspense>
```

Skeleton screens are preferable when the shape of the resolved content is known and stable. Spinners are sufficient for transient, global, or low-visibility operations where layout continuity isn't a concern.

✓ Tip

Skeletons, like any other React components, are subject to client-side hydration. If skeleton structures are themselves complex, they can carry a non-trivial rendering cost. For content outside the initial viewport, this cost can outweigh the benefit: users don't perceive layout shift for content they can't yet see, and the skeleton itself contributes to the hydration workload



Global vs. Local Loading Indicators

A global loading indicator communicates the state of the app as a whole, typically a progress bar or spinner in the navigation area. It's appropriate for route transitions where the entire page content is being replaced.

```
1 // in a root or layout component
2 const navigation = useNavigation();
3
4 return (
5   <>
6     {navigation.state === "loading" && <ProgressBar
7       <Outlet />
8     </>
9   );
```

A local loading indicator is scoped to the specific UI region affected by the operation. It's appropriate for partial updates, such as deferred data within a route or out-of-band mutations via `useFetcher()`, where the rest of the UI remains interactive.

```
1 const fetcher = useFetcher();
2
3 return (
4   <section>{fetcher.state === "loading" ? <Skeleton
5   );
```

Global and local indicators aren't mutually exclusive. A route transition can warrant both a global progress indicator and local skeleton screens for individual content regions.

Preventing Layout Shift

Layout shift occurs when content renders into a space that wasn't reserved for it, causing surrounding elements to reposition. Layout shift degrades both visual stability and user-perceived performance. It's measured by the [Cumulative Layout Shift \(CLS\)](#) [↗](#) metric.

The primary mitigation is to reserve space for loading content before it resolves. Skeleton screens are the most direct approach, but explicit dimension constraints on container elements achieve the same effect when a skeleton isn't warranted.

```
1 // reserve space via explicit dimensions
2 <div style={{ minHeight: "200px" }}>
3   <Suspense fallback={null}>
4     <Await resolve={data}>{(resolved) => <Component
5   </Suspense>
6 </div>
```

[Try for free](#)

Optimistic UI: No Loading States

Optimistic UI skips the loading state entirely by assuming that an operation will succeed and updating the UI immediately. The actual server response either confirms the update or triggers a rollback. This is the most responsive pattern available, but introduces complexity around error handling and state consistency.

Optimistic UI is appropriate when the probability of failure is low and the operation is reversible. It's not suitable for destructive or irreversible actions where a failed rollback leaves the UI in an inconsistent state.

`useFetcher()` exposes submitted form data via `fetcher.formData` before the action completes, enabling optimistic updates without external state management. For implementation patterns using `fetcher.formData` and `useOptimistic`, see [State Management](#).

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

[Heroku](#)
[MuleSoft](#)
[Tableau](#)
[Commerce Cloud](#)
[Lightning Design System](#)
[Einstein](#)
[Quip](#)

POPULAR RESOURCES

[Documentation](#)
[Component Library](#)
[APIs](#)
[Trailhead](#)
[Sample Apps](#)
[AppExchange](#)

COMMUNITY

[Trailblazer Community](#)
[Events and Calendar](#)
[Partner Community](#)
[Blog](#)
[Salesforce Admins](#)
[Salesforce Architects](#)

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

